

FASE 0 · O AGENTE EM CÓDIGO PURO

Aula 0.2

Tool use, o mecanismo cru

O modelo nunca executa nada. Ele pede, você executa, você devolve.
O ciclo de quatro etapas na mão, nos dois providers.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção

Projeto: Núcleo de Agentes Operacionais

Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código

Pré-requisito: Aula 0.1 concluída (tag **aula-0.1**)

1. Objetivo de aprendizagem

Ao final desta aula você será capaz de declarar uma ferramenta, fazer o modelo **pedir** para usá-la, **executar** a função no seu código e **devolver** o resultado para o modelo produzir a resposta final, entendendo o ciclo cru nos dois providers, sem nenhuma abstração escondendo o que acontece.

2. Por que esta aula importa

Um agente é, no fundo, um modelo que sabe pedir ajuda ao seu código. Tudo o que vem nas próximas fases, o loop, o multi-agente, o MCP, é este ciclo repetido e organizado. Quando um agente em produção chama a ferramenta errada, ignora uma ferramenta ou trava, você só sabe onde olhar se já escreveu cada etapa na mão. Depois de hoje, você diagnostica qual das quatro etapas falhou.

É também o primeiro ponto onde os providers divergem de verdade. Entender onde eles diferem e o que é igual por baixo é o que torna o seu código portátil, que é regra central desta série.

3. Teoria

3.1 A lógica universal: o modelo não executa nada

Desfaça já um mal-entendido comum. O modelo **nunca executa a sua função**. Ele não roda código, não toca no seu banco, não chama a sua API. Ele gera um texto estruturado dizendo: eu gostaria de chamar a ferramenta X com estes argumentos.

Quem executa é você. O ciclo tem quatro etapas, sempre as mesmas, em qualquer provider ou framework:

Etapa	Quem faz	O que acontece
1. Declarar	Você	Envia, junto com as mensagens, a lista de ferramentas: nome, descrição e schema dos parâmetros.
2. Pedir	O modelo	Em vez de texto, responde com um pedido estruturado: qual ferramenta, quais argumentos, e um id daquele pedido.
3. Executar	Você	Roda a função Python correspondente, no seu código.
4. Devolver	Você	Acrescenta à conversa o pedido do modelo e o resultado, amarrados pelo id, e chama de novo.

GRAVE ISTO

O modelo decide, você age. Essa separação é a alma do agente. É também o que torna segurança possível, já que nada acontece sem passar pelo seu código.

3.2 O schema é um contrato, não um detalhe

Você descreve a ferramenta com um **JSON Schema**: um objeto com o nome, o que ela faz, e quais parâmetros aceita, cada um com tipo e obrigatoriedade.

ANALOGIA

O schema é o cardápio. O modelo é o cliente. Ele só pede o que está escrito, usando os nomes exatos do cardápio. Se a descrição do prato é vaga, o cliente pede errado. Por isso o texto da `description` não é enfeite: é ele que o modelo lê para decidir quando e como usar a ferramenta. Design de ferramenta é o tema da Aula 0.5; a semente é essa.

3.3 Onde Anthropic e OpenAI concordam e divergem

O ciclo de quatro etapas é idêntico. O formato muda em três pontos concretos:

	Anthropic	OpenAI
Onde mora o schema	<code>name</code> , <code>description</code> e <code>input_schema</code> no topo da ferramenta	tudo aninhado em <code>function</code> , e o schema chama <code>parameters</code>
Como chega o pedido	bloco <code>tool_use</code> com <code>id</code> , <code>name</code> e <code>input</code> ; o input já vem como dicionário	lista <code>tool_calls</code> com <code>id</code> , <code>function.name</code> e <code>function.arguments</code> ; o arguments vem como string JSON , precisa de <code>json.loads</code>
Como você devolve	mensagem <code>role: "user"</code> com bloco <code>tool_result</code> apontando o <code>tool_use_id</code>	mensagem <code>role: "tool"</code> com <code>tool_call_id</code> apontando o pedido

Escreva esses três pontos num papel. São exatamente eles que a abstração da Aula 0.4 vai esconder. Hoje deixamos tudo cru de propósito: você precisa sentir a diferença antes de escondê-la.

3.4 Por que uma volta, e não o loop inteiro

Hoje fazemos **uma volta**: o modelo pede, você executa, devolve, ele responde, fim. Não automatizamos a repetição ainda. Isso é a Aula 0.3. A razão é pedagógica: se você domina uma volta manual, o `while` da próxima aula é só repetir isso até o modelo parar de pedir ferramentas.

4. Mão na massa

O código de hoje nasce como **teste versionado** em `tests/test_tooluse.py`, consumindo o `config.py` da Aula 0.1. A ferramenta mock e os schemas ficam ali como material didático; as três ferramentas reais do Núcleo chegam na Aula 0.5. Para explorar célula a célula enquanto estuda, use um rascunho em `scratch/`, que o `.gitignore` já ignora: só entra no repositório o que é produto ou o que testa o produto.

Uma nota antes de começar: estes testes batem na **API real**. Eles custam alguns centavos de token e exigem rede. Por isso levam um marcador próprio e só rodam quando você chama de propósito, o que o Passo 5 organiza.

Passo 1 — A ferramenta de verdade (a função Python)

O que e por quê: antes de qualquer schema, a ferramenta é Python comum. Ela nem sabe que existe um modelo. A lógica de negócio vive separada do LLM.

```
# tests/test_tooluse.py
import json

import pytest
from dotenv import load_dotenv

# O conftest.py injeta chaves FALSAS para a suite rapida rodar sem
# configuracao. Os testes de integracao precisam das chaves REAIS,
# entao recarregamos o .env com override=True, que substitui as
# falsas se o arquivo existir.
load_dotenv(override=True)

def buscar_info_lead(email: str) -> dict:
    """Busca dados de um lead pela chave de email.
    Mock: em producao seria uma consulta a CRM ou banco."""
    base_fake = {
        "ana@techflow.com": {
            "nome": "Ana Prado", "empresa": "TechFlow",
            "plano": "trial", "mrr": 0,
        },
        "carlos@datalog.io": {
            "nome": "Carlos Nunes", "empresa": "DataLog",
            "plano": "pro", "mrr": 490,
        },
    }
    return base_fake.get(email, {"erro": "lead nao encontrado"})
```

Resultado esperado: `buscar_info_lead("carlos@datalog.io")` retorna o dicionário do Carlos. Repare no bloco do `load_dotenv`: é a ponte entre a infraestrutura de teste da Aula 0.1 (que injeta chaves falsas para a suíte rápida) e os testes de hoje, que precisam das reais.

Passo 2 — O schema nos dois formatos, como fixtures

O que e por quê: o mesmo contrato, escrito do jeito de cada provider. A duplicação incomoda, e é de propósito: ela é o argumento da Aula 0.4.

```

@pytest.fixture
def tool_anthropic():
    # Anthropic: name, description e input_schema no topo.
    return {
        "name": "buscar_info_lead",
        "description": (
            "Busca os dados cadastrais de um lead a partir do email. "
            "Use quando precisar saber empresa, plano ou MRR de um lead."
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "email": {
                    "type": "string",
                    "description": "Email do lead, ex: nome@empresa.com",
                }
            },
            "required": ["email"],
        },
    }

@pytest.fixture
def tool_openai():
    # OpenAI: tudo aninhado em "function"; o schema chama "parameters".
    return {
        "type": "function",
        "function": {
            "name": "buscar_info_lead",
            "description": (
                "Busca os dados cadastrais de um lead a partir do email. "
                "Use quando precisar saber empresa, plano ou MRR de um lead."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "email": {
                        "type": "string",
                        "description": "Email do lead, ex: nome@empresa.com",
                    }
                },
                "required": ["email"],
            },
        },
    }

```

O que faz: descreve a mesma ferramenta duas vezes. O ponto 1 da teoria à vista: `input_schema` no topo contra `parameters` dentro de `function`. O miolo (`properties`, `required`) é **idêntico**. Isso é o portátil.

Passo 3 — O ciclo completo na Anthropic

O que e por quê: um teste de integração que faz a volta inteira e prova que o modelo usou o resultado da ferramenta.

```

@pytest.mark.integration
def test_ciclo_tool_use_anthropic(tool_anthropic):
    from anthropic import Anthropic

    from nucleo import config

    client = Anthropic() # le ANTHROPIC_API_KEY do ambiente
    pergunta = "Qual e o plano do lead carlos@datalog.io?"
    msgs = [{"role": "user", "content": pergunta}]

    # 1a chamada: o modelo PEDE a ferramenta (etapas 1 e 2).
    resp1 = client.messages.create(
        model=config.ANTHROPIC_MODEL,
        max_tokens=1024, # obrigatorio na Anthropic
        tools=[tool_anthropic],
        messages=msgs,
    )
    assert resp1.stop_reason == "tool_use"

    tool_block = next(b for b in resp1.content if b.type == "tool_use")
    assert tool_block.name == "buscar_info_lead"
    # Na Anthropic o input JA vem como dicionario Python.
    assert tool_block.input == {"email": "carlos@datalog.io"}

    # Etapa 3: VOCE executa a funcao de verdade.
    resultado = buscar_info_lead(**tool_block.input)

    # Etapa 4: devolve o pedido do modelo + o resultado, pareados
    # pelo tool_use_id, numa mensagem de role "user".
    msgs.append({"role": "assistant", "content": resp1.content})
    msgs.append({
        "role": "user",
        "content": [{
            "type": "tool_result",
            "tool_use_id": tool_block.id,
            "content": json.dumps(resultado),
        }],
    })

    # 2a chamada: agora o modelo RESPONDE em texto.
    resp2 = client.messages.create(
        model=config.ANTHROPIC_MODEL,
        max_tokens=1024,
        tools=[tool_anthropic],
        messages=msgs,
    )
    texto = resp2.content[0].text.lower()
    # "pro" e "datalog" so existem no mock: se aparecem, o modelo
    # leu o resultado da ferramenta, nao inventou.
    assert "pro" in texto or "datalog" in texto

```

O que faz: percorre as quatro etapas e afirma cada uma. Repare na etapa 4: você reinsere `resp1.content` como mensagem do assistant e adiciona um user com `tool_result` referenciando o `tool_use_id`. Sem esse pareamento, a API rejeita. A asserção final prova que o modelo leu o resultado, já que pro e DataLog só existem no mock.

Passo 4 — O mesmo ciclo na OpenAI, sentindo as diferenças

O que e por quê: repetir na OpenAI fixa o que é universal e o que é formato. As três diferenças aparecem no código.

```

@pytest.mark.integration
def test_ciclo_tool_use_openai(tool_openai):
    from openai import OpenAI

    from nucleo import config

    client = OpenAI() # le OPENAI_API_KEY do ambiente
    pergunta = "Qual e o plano do lead carlos@datalog.io?"
    msgs = [{"role": "user", "content": pergunta}]

    r1 = client.chat.completions.create(
        model=config.OPENAI_MODEL,
        tools=[tool_openai],
        messages=msgs,
    )
    msg = r1.choices[0].message
    tool_call = msg.tool_calls[0]
    assert tool_call.function.name == "buscar_info_lead"

    # DIFERENCA 2: arguments vem como STRING JSON, nao como dict.
    args = json.loads(tool_call.function.arguments)
    resultado = buscar_info_lead(**args)

    # DIFERENCA 3: devolve com role "tool" e tool_call_id.
    msgs.append(msg) # a mensagem do assistant com o tool_call
    msgs.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": json.dumps(resultado),
    })

    r2 = client.chat.completions.create(
        model=config.OPENAI_MODEL,
        tools=[tool_openai],
        messages=msgs,
    )
    texto = r2.choices[0].message.content.lower()
    assert "pro" in texto or "datalog" in texto

```

O que faz: mesmo ciclo, três diferenças na prática: os argumentos vêm como string e exigem `json.loads`; a devolução usa `role: "tool"` com `tool_call_id`; e o teto de saída não é obrigatório aqui.

Passo 5 — Separar a suíte rápida da suíte que custa dinheiro

O que e por quê: os testes de hoje custam token e dependem de rede. Eles não podem rodar toda vez que você digita `pytest`, nem no CI da Fase 3 sem intenção. Dois movimentos no `pytest.ini` resolvem: registrar o marcador e **excluir por padrão** os testes marcados.

```

# pytest.ini (na raiz do projeto)
[pytest]
markers =
    integration: testes que batem em API real (custam token, exigem rede)
# A suite rapida nao roda os testes de integracao por padrao.
# Para roda-los de proposito: pytest -m integration
addopts = -m "not integration"

```

Atenção ao detalhe: registrar o marcador sozinho não exclui nada, só elimina o aviso de marcador desconhecido. Quem faz a exclusão é o `addopts`. E quando você chama `pytest -m`

`integration` na linha de comando, o seu `-m` vence o do arquivo, e só os testes de integração rodam.

Resultado esperado: `pytest` sozinho mostra `5 passed, 2 deselected`. `pytest -m integration` roda os dois testes de hoje contra as APIs reais e ambos passam.

Passo 6 — Commit e tag

```
git status          # .env NAO pode aparecer
git add .
git commit -m "Aula 0.2: tool use cru nos dois providers"
git tag aula-0.2
git push && git push --tags
```

5. Portabilidade

O que você escreveu hoje reaparece em toda parte, porque **todo framework de agente é uma embalagem em cima deste ciclo de quatro etapas**.

- No **LangGraph** (Fase 1), o `ToolNode` faz as etapas 3 e 4 por você: recebe o pedido, executa, devolve como observação. Você não escreve mais o `tool_result` na mão, mas sabe que é isso por baixo.
- No **MCP** (Fase 2), este mesmo JSON Schema vira o contrato padronizado que qualquer agente entende. O `input_schema` de hoje é a base do que o MCP formaliza.
- O `tool_choice` (forçar ou proibir uma ferramenta) existe nos dois providers com sintaxes diferentes, e todo framework o expõe. Mesmo conceito.

A lição portátil: **o modelo pede, você executa, você devolve**. Troque o provider, troque o framework, o esqueleto não muda. Só muda quem escreve o boilerplate.

6. Terminei quando

O critério da aula: **o modelo chama a minha ferramenta e usa o resultado na resposta final, nos dois providers**.

- `pytest -m integration` roda os dois testes e ambos passam.
- A suíte rápida (`pytest` sem flag) continua verde e não dispara os testes de hoje, provando que a separação funciona.
- Troque a pergunta para `ana@techflow.com` e confirme que a resposta refletiria trial. Isso mostra que o modelo lê o resultado, não inventa.

7. Erros comuns e troubleshooting

- **authentication_error só nos testes de integração**. O conftest injetou a chave falsa e o `load_dotenv(override=True)` não está no topo do arquivo, ou o `.env` não está na raiz. É a interação entre a infraestrutura da 0.1 e os testes de hoje.
- **Esquecer o `json.loads` na OpenAI**. O arguments é string; passar direto para `**args` quebra. Na Anthropic, o input já é dict: não use `json.loads` lá.

- **A API da Anthropic rejeita a devolução.** Você não reinseriu o `respl.content` como mensagem do assistant antes do `tool_result`, ou o `tool_use_id` não bate com o id do pedido.
- **Trocar `role: "tool"` por `role: "user"`.** São os formatos de devolução de cada provider. Não misture: é o erro mais comum de quem vem de um provider só.
- **Aviso de marcador desconhecido.** Faltou registrar o `integration` no `pytest.ini`.
- **A suíte rápida disparou os testes de integração.** Faltou o `addopts = -m "not integration"`: registrar o marcador sozinho não exclui nada.
- **O modelo não chama a ferramenta.** Quase sempre a description está vaga. Melhore-a. Aprofundamos na Aula 0.5.
- **model not found.** O nome no `.env` não existe na sua conta. Valide na doc da Anthropic ou no painel da OpenAI.

8. Ponte para a próxima aula

Hoje demos uma volta na mão. Na **Aula 0.3, O loop de agente**, você troca o passo manual por um `while` que encadeia ferramentas até o modelo parar de pedir, com condição de parada e limite de iteração. Mesma mecânica de hoje, agora repetida por conta própria. É a aula em que o projeto deixa de ser uma chamada de API e vira um agente.